Платформено-независими програмни езици

Лекция 4

Интерфейси, вложени класове и ламбда изрази

Договори между класове, вложени и анонимни класове, функционални интерфейси, анотации и JavaDoc

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1. Интерфейси
2. Методи по подразбиране и функционални интерфейси
3. Вложени класове
4. Ламбда изрази
5. Анотации и документация

01

Интерфейси

Договор за поведение

Какво е интерфейс?

- В реалния свят: **начин на взаимодействие** – напр. контактът и щепселът
- В програмирането: **договор** – какво може да прави обектът, без да казва как
- Класът, който **имплементира** интерфейса, се задължава да реализира методите му
- Различни, несвързани класове могат да имат общ интерфейс
- Кодът работи с интерфейса, а не с конкретната реализация

Деклариране на интерфейс

- Ключова дума `interface`
- Методите са `public abstract` по подразбиране
- Полетата са `public static final` – константи
- Не може да се създаде обект на интерфейс

```
public interface Shape {
    double PI_APPROX = 3.14;       // константа

    double area();                 // абстрактен
    double perimeter();            // абстрактен
}
```

Реализация на интерфейс

- `class X implements Интерфейс`
- Класът трябва да реализира **всички** абстрактни методи
- Иначе класът трябва да е `abstract`
- Методите се реализират като `public`

```
public class Circle implements Shape {
    private final double r;

    public Circle(double r) {
        this.r = r;
    }

    @Override
    public double area() {
        return Math.PI * r * r;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * r;
    }
}
```

Интерфейсът като тип

- Променлива, параметър или резултат може да е от тип интерфейс
- Може да сочи обект от **всеки** клас, който го имплементира
- Полиморфизъм без обща йерархия от класове

```
static double totalArea(List<Shape> shapes) {
    double sum = 0;
    for (Shape s : shapes) {
        sum += s.area();
    }
    return sum;
}

List<Shape> list = List.of(
        new Circle(1),
        new Rectangle(2, 3));

System.out.println(totalArea(list));
```

Пример: начини на плащане

- Интерфейсът описва **какво** – „може да плати“
- Всеки клас решава **как**
- Магазинът не зависи от конкретния начин – лесно добавяме нов

```
interface PaymentMethod {
    boolean pay(double amount);
}

class CardPayment implements PaymentMethod {
    public boolean pay(double amount) {
        return bank.charge(amount);
    }
}

class CashPayment implements PaymentMethod {
    public boolean pay(double amount) {
        return drawer.receive(amount);
    }
}

void checkout(PaymentMethod pm, double sum) {
    pm.pay(sum);
}
```

Няколко интерфейса наведнъж

- Класът наследява **един** клас, но имплементира **много** интерфейси
- Това е начинът в Java за „множествено наследяване“ на тип
- Интерфейсите могат да наследяват други интерфейси (`extends`)

```
interface Printable {
    void print();
}

interface Saveable {
    void save(String file);
}

class Report extends Document
        implements Printable, Saveable {

    public void print() { /* ... */ }

    public void save(String file) { /* ... */ }
}

interface Persistable
        extends Saveable, Printable { }
```

Интерфейси от стандартната библиотека

Интерфейс	Метод	Предназначение
`Comparable<T>`	`compareTo(T o)`	естествена наредба – за сортиране
`Comparator<T>`	`compare(T a, T b)`	външна наредба
`Iterable<T>`	`iterator()`	обхождане с `for-each`
`Runnable`	`run()`	задача за изпълнение (напр. в нишка)
`AutoCloseable`	`close()`	ресурс в `try-with-resources`
`CharSequence`	`length()`, `charAt()`	последователност от символи

Пример: Comparable

```
public record Student(String name, double grade)
        implements Comparable<Student> {

    @Override
    public int compareTo(Student other) {
        return Double.compare(other.grade, this.grade);   // по оценка, низходящо
    }
}

List<Student> list = new ArrayList<>(List.of(
        new Student("Ана", 5.50),
        new Student("Боян", 6.00),
        new Student("Вера", 4.75)));

Collections.sort(list);
System.out.println(list.get(0).name());   // Боян
```

`compareTo` връща отрицателно число, 0 или положително – „по-малък“, „равен“, „по-голям“

Собствена колекция с for-each

- `Iterable<T>` → обектът може да се обхожда с `for-each`
- Итераторът може да е анонимен клас
- Пример: диапазон от числа като в Python `range`

```
record Range(int from, int to)
        implements Iterable<Integer> {

    public Iterator<Integer> iterator() {
        return new Iterator<>() {
            int cur = from;
            public boolean hasNext() {
                return cur < to;
            }
            public Integer next() {
                return cur++;
            }
        };
    }
}

for (int i : new Range(1, 5)) {
    System.out.print(i + " ");   // 1 2 3 4
}
```

Абстрактен клас или интерфейс?

Абстрактен клас

- може да има състояние (полета)
- конструктори
- само **един** родител
- обща част от реализацията
- връзка „е“ в тясно свързана йерархия

Интерфейс

- без състояние (само константи)
- без конструктори
- класът може да имплементира **много**
- описва **способност**: `Comparable`, `Runnable`
- свързва несвързани класове

Кога какво използваме?

- **Интерфейс** – когато описваме роля или способност („може да се сравнява“, „може да се затваря“)
- **Абстрактен клас** – когато наследниците споделят код и състояние
- Често се комбинират: интерфейс + абстрактен клас с частична реализация
  - `List` → `AbstractList` → `ArrayList`
- Правило: програмирайте към **интерфейса**, а не към реализацията
  - `List<String> names = new ArrayList<>();`

Интерфейси в Python и в Java

Python

- „duck typing“: ако има метод `quack()`, значи е патица
- проверката е при **изпълнение**
- формални интерфейси – `abc.ABC` и `@abstractmethod`
- `typing.Protocol` – структурни типове

Java

- класът **обявява** `implements Интерфейс`
- проверката е при **компилиране**
- липсващ метод → грешка при компилиране
- интерфейсът е тип: параметри, полета, резултати

02

Методи по подразбиране и функционални интерфейси

Еволюция на интерфейсите от Java 8 насам

Методи по подразбиране (default)

- От Java 8 интерфейсът може да има методи **с тяло**
- Позволяват да добавим метод към интерфейс, без да счупим съществуващите класове
- Класът може да ги припокрие

```
public interface Shape {
    double area();
    double perimeter();

    default String describe() {
        return "%s: S=%.2f, P=%.2f".formatted(
                getClass().getSimpleName(),
                area(), perimeter());
    }
}

Shape c = new Circle(1);
System.out.println(c.describe());
// Circle: S=3.14, P=6.28
```

Статични и private методи в интерфейс

- `static` метод – помощна функция, извиква се през интерфейса
- `private` метод (Java 9) – общ код за default методите
- Пример от JDK: `List.of(...)`, `Comparator.comparing(...)`

```
public interface Shape {
    double area();

    static Shape square(double a) {
        return new Rectangle(a, a);
    }

    default boolean isLarge() {
        return area() > limit();
    }

    private double limit() {
        return 100.0;
    }
}

Shape sq = Shape.square(12);
System.out.println(sq.isLarge());  // true
```

Конфликт между default методи

- Ако два интерфейса дават default метод с едно и също име – **конфликт**
- Класът е длъжен да го припокрие
- `Интерфейс.super.метод()` – избира конкретна версия
- Правило: методът на класа винаги печели пред default метод

```
interface Walker {
    default String move() { return "ходя"; }
}

interface Swimmer {
    default String move() { return "плувам"; }
}

class Duck implements Walker, Swimmer {
    @Override
    public String move() {
        return Walker.super.move() + " и "
             + Swimmer.super.move();
    }
}

new Duck().move();   // ходя и плувам
```

Промяна на интерфейс

- Интерфейсът е **договор** – промяната засяга всички реализации
- Добавяне на абстрактен метод → всички класове спират да се компилират
- Решения:
  - нов метод като `default`
  - нов интерфейс, който наследява стария

```java
public interface DoIt {
    void doSomething(int i, double x);
}

// Вариант 1: default метод
public interface DoIt {
    void doSomething(int i, double x);

    default boolean didItWork() {
        return true;
    }
}

// Вариант 2: нов интерфейс
public interface DoItPlus extends DoIt {
    boolean didItWork();
}
```

Функционални интерфейси

- Интерфейс с **точно един** абстрактен метод
- `@FunctionalInterface` – компилаторът проверява това
- Може да се реализира с **ламбда израз**
- Основата на функционалния стил в Java

```java
@FunctionalInterface
public interface Operation {
    double apply(double a, double b);
}

Operation add = (a, b) -> a + b;
Operation mul = (a, b) -> a * b;

System.out.println(add.apply(3, 4));  // 7.0
System.out.println(mul.apply(3, 4));  // 12.0
```

Стандартни функционални интерфейси

Интерфейс	Метод	Описание	Пример
`Function<T,R>`	`R apply(T t)`	преобразува T в R	`s -> s.length()`
`Predicate<T>`	`boolean test(T t)`	условие	`n -> n > 0`
`Consumer<T>`	`void accept(T t)`	използва стойност	`s -> System.out.println(s)`
`Supplier<T>`	`T get()`	доставя стойност	`() -> new ArrayList<>()`
`UnaryOperator<T>`	`T apply(T t)`	T → T	`s -> s.toUpperCase()`
`BinaryOperator<T>`	`T apply(T a, T b)`	(T, T) → T	`(a, b) -> a + b`

03

Вложени класове

Статични вложени, вътрешни, локални и анонимни класове

Видове вложени класове

Вид	Деклариране	Достъп до обекта на външния клас
статичен вложен (static nested)	`static class` в тялото на клас	не – само до статичните членове
вътрешен (inner)	`class` в тялото на клас	да – има скрита референция
локален (local)	в тялото на метод	да + ефективно `final` локални променливи
анонимен (anonymous)	`new Интерфейс() { ... }`	да + ефективно `final` локални променливи

Защо се използват вложени класове?

- **Логическо групиране** – класът се използва само на едно място
- **По-добро капсулиране** – вложеният клас вижда `private` членовете на външния
- **По-четим код** – помощният клас е до кода, който го използва
- Примери от JDK:
  - `Map.Entry` – статичен вложен интерфейс
  - итераторите на колекциите – вътрешни класове
- Внимание: прекаленото влагане прави кода труден за четене

Статичен вложен клас

- Членува във външния клас, но **не зависи** от негов обект
- Създава се без обект на външния клас: `Outer.Nested`
- Вижда само статичните членове на външния клас

```
public class Geometry {

    public static class Point {
        final double x, y;

        public Point(double x, double y) {
            this.x = x;
            this.y = y;
        }
    }

    public static double dist(Point a, Point b) {
        return Math.hypot(a.x - b.x, a.y - b.y);
    }
}

var p = new Geometry.Point(0, 0);
var q = new Geometry.Point(3, 4);
Geometry.dist(p, q);     // 5.0
```

Статичен вложен клас: шаблонът Builder

```
public class Pizza {
    private final String size;
    private final boolean cheese;

    private Pizza(Builder b) { size = b.size; cheese = b.cheese; }

    public static class Builder {
        private String size = "M";
        private boolean cheese;

        public Builder size(String s) { size = s; return this; }
        public Builder cheese()       { cheese = true; return this; }
        public Pizza build()          { return new Pizza(this); }
    }
}
```

Вложеният `Builder` вижда `private` конструктора на `Pizza` – само той може да създава пици

Използване на Builder

- Четим код при много незадължителни параметри
- Обектът е неизменяем след `build()`
- Шаблонът се среща често в библиотеките (напр. `HttpRequest.newBuilder()`)

```
Pizza p = new Pizza.Builder()
        .size("L")
        .cheese()
        .build();

HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create("https://dev.java"))
        .GET()
        .build();
```

Вътрешен (inner) клас

```
public class Playlist implements Iterable<String> {
    private final String[] songs = {"Песен 1", "Песен 2", "Песен 3"};

    @Override
    public Iterator<String> iterator() {
        return new PlaylistIterator();
    }

    private class PlaylistIterator implements Iterator<String> {
        private int index = 0;

        public boolean hasNext() { return index < songs.length; }  // songs!
        public String next() { return songs[index++]; }
    }
}

for (String s : new Playlist()) {
    System.out.println(s);
}
```

Обект на вътрешен клас

- Вътрешният обект винаги е свързан с **обект на външния клас**
- Отвън се създава със синтаксиса `външен.new Вътрешен()`
- `Outer.this` – референция към външния обект

```
public class Bank {
    private String name = "ТУ Банк";

    public class Branch {
        private final String city;

        public Branch(String city) { this.city = city; }

        public String info() {
            return Bank.this.name + ", " + city;
        }
    }
}

Bank bank = new Bank();
Bank.Branch b = bank.new Branch("София");
System.out.println(b.info());
// ТУ Банк, София
```

Локален клас

- Деклариран вътре в метод
- Видим само в този метод
- Може да използва локалните променливи, ако са **ефективно final** (не се променят)

```
static List<String> validate(String email) {
    List<String> errors = new ArrayList<>();
    int max = 50;

    class Check {
        void rule(boolean ok, String msg) {
            if (!ok) errors.add(msg);
        }
    }

    Check c = new Check();
    c.rule(email.contains("@"), "Липсва @");
    c.rule(email.length() <= max,
           "Твърде дълъг");
    return errors;
}
```

Анонимен клас

- Клас без име, деклариран и инстанциран **на едно място**
- Реализира интерфейс или наследява клас
- Използва се за еднократна реализация
- Преди Java 8 – основният начин за „предаване на поведение“

```
Comparator<String> byLength =
        new Comparator<String>() {
            @Override
            public int compare(String a, String b) {
                return Integer.compare(
                        a.length(), b.length());
            }
        };

List<String> words = new ArrayList<>(
        List.of("Java", "C", "Python"));
words.sort(byLength);
// [C, Java, Python]
```

Обратно извикване (callback)

- Обект получава интерфейс и го „вика“ при събитие
- Основата на събитийното програмиране
- Наблюдател (observer): много слушатели за едно събитие

```
interface PriceListener {
    void onChange(String item, double price);
}

class Shop {
    private final List<PriceListener> listeners =
            new ArrayList<>();

    void subscribe(PriceListener l) {
        listeners.add(l);
    }

    void setPrice(String item, double p) {
        listeners.forEach(l -> l.onChange(item, p));
    }
}

shop.subscribe((i, p) ->
        System.out.println(i + " -> " + p));
```

04

Ламбда изрази

Предаване на поведение като стойност (Java 8+)



Ламбда изразът е кратък запис на анонимна функция

Реализира функционален интерфейс без излишния синтаксис на анонимния клас. Подобно на `lambda` в Python, но тялото може да има и няколко оператора.

Анонимен клас срещу ламбда

Анонимен клас

```
new Comparator<String>() {
  public int compare(String a, String b) {
    return a.length() - b.length();
  }
}
```

Ламбда израз

```
(a, b) -> a.length() - b.length()
```

- типовете на параметрите се извеждат
- без `return` и `{ }` при един израз
- **5 реда → 1 ред**

Ламбда в Python и в Java

Python

- `square = lambda x: x * x`
- `sorted(words, key=len)`
- `filter(lambda n: n > 0, nums)`
- функцията е обект от тип `function`

Java

- `Function<Integer,Integer> square = x -> x * x;`
- `words.sort(Comparator.comparing(String::length));`
- `nums.removeIf(n -> n <= 0);`
- ламбдата реализира **функционален интерфейс**

Синтаксис на ламбда изразите

Форма	Пример
без параметри	`() -> System.out.println("Здравей")`
един параметър	`x -> x * x`
няколко параметъра	`(a, b) -> a + b`
с явни типове	`(String s, int n) -> s.repeat(n)`
с блок	`(a, b) -> { int s = a + b; return s * 2; }`
с `var`	`(var a, var b) -> a + b`

Ламбда изрази в действие

```
List<String> names = new ArrayList<>(List.of("Мария", "Иван", "Ана", "Георги"));

// Сортиране по дължина
names.sort((a, b) -> Integer.compare(a.length(), b.length()));

// Обхождане
names.forEach(n -> System.out.println("Здравей, " + n));

// Премахване по условие
names.removeIf(n -> n.length() < 4);

// Замяна на всеки елемент
names.replaceAll(n -> n.toUpperCase());

System.out.println(names);   // [ИВАН, МАРИЯ, ГЕОРГИ]
```

Затваряне (capture) на променливи

- Ламбдата може да използва локални променливи от обкръжението
- Те трябва да са **ефективно final**
- Полетата на обекта могат да се променят

```
int limit = 10;
Predicate<Integer> big = n -> n > limit;

System.out.println(big.test(15)); // true

// limit = 20;
// грешка: limit вече не е
// ефективно final

int[] counter = {0};   // заобикаляне
names.forEach(n -> counter[0]++);
```

this в анонимен клас и в ламбда

Анонимен клас

- създава **нов обект**
- `this` сочи анонимния обект
- може да има собствени полета
- може да реализира интерфейс с много методи

Ламбда израз

- **не** създава нов обхват за `this`
- `this` сочи обграждащия обект
- без състояние
- само за функционални интерфейси

Референции към методи

Вид	Синтаксис	Еквивалентна ламбда
статичен метод	`Integer::parseInt`	`s -> Integer.parseInt(s)`
метод на конкретен обект	`System.out::println`	`x -> System.out.println(x)`
метод на произволен обект от типа	`String::toUpperCase`	`s -> s.toUpperCase()`
конструктор	`ArrayList::new`	`() -> new ArrayList<>()`

Функционални интерфейси за примитивни типове

Интерфейс	Метод	Предимство
`IntPredicate`	`boolean test(int x)`	без опаковане в `Integer`
`IntFunction<R>`	`R apply(int x)`	без опаковане
`ToIntFunction<T>`	`int applyAsInt(T t)`	връща примитив
`IntUnaryOperator`	`int applyAsInt(int x)`	`x -> x * 2`
`IntBinaryOperator`	`int applyAsInt(int a, int b)`	`Math::max`
`DoubleSupplier`	`double getAsDouble()`	`Math::random`

Comparator с ламбда изрази и референции

```
record Student(String name, int course, double grade) { }

List<Student> list = new ArrayList<>(List.of(
        new Student("Ана", 2, 5.5),
        new Student("Боян", 1, 6.0),
        new Student("Вера", 2, 6.0)));

// по оценка, низходящо, при равенство – по име
list.sort(Comparator.comparingDouble(Student::grade)
                    .reversed()
                    .thenComparing(Student::name));

// по курс, после по име
list.sort(Comparator.comparingInt(Student::course)
                    .thenComparing(Student::name));
```

`Comparator.comparing` + референции към методи = четимо сортиране по няколко критерия

Композиция на функции

- Функционалните интерфейси имат default методи за **комбиниране**
- `Predicate`: `and`, `or`, `negate`
- `Function`: `andThen`, `compose`
- Изграждаме сложна логика от прости части

```
Predicate<String> notEmpty = s -> !s.isBlank();
Predicate<String> isShort = s -> s.length() < 10;
Predicate<String> valid = notEmpty.and(isShort);

valid.test("Java");         // true
valid.test("   ");          // false

Function<String, String> trim = String::strip;
Function<String, Integer> len = String::length;
Function<String, Integer> f = trim.andThen(len);

f.apply("  Java  ");        // 4
```

Supplier и мързеливо изчисление

- `Supplier<T>` – „рецепта“ за стойност, изчислява се при `get()`
- Изчисляваме само **ако** стойността е нужна
- Пример от JDK: `Optional.orElseGet`, логерите (лекция 9)

```
static String expensiveReport() {
    System.out.println("Генерирам...");
    return "отчет";
}

static void log(boolean debug,
                Supplier<String> msg) {
    if (debug) {
        System.out.println(msg.get());
    }
}

log(false, () -> expensiveReport());
// нищо не се генерира!

log(true, Main::expensiveReport);
// Генерирам...  отчет
```

Собствени методи, които приемат поведение

- Параметър от функционален тип = „дупка“ за поведение
- Извикващият подава ламбда
- Един метод – много приложения

```
static <T> List<T> filter(List<T> list,
                          Predicate<T> p) {
    List<T> result = new ArrayList<>();
    for (T x : list) {
        if (p.test(x)) result.add(x);
    }
    return result;
}

var nums = List.of(3, -1, 8, 0, -5, 12);
filter(nums, n -> n > 0);      // [3, 8, 12]
filter(nums, n -> n % 2 == 0); // [8, 0, 12]
```

Ламбда изрази и нишки

- `Runnable` е функционален интерфейс: `void run()`
- Задача за нишка – с ламбда
- Подробно – в лекцията за нишки

```
Runnable task = () -> {
    for (int i = 1; i <= 3; i++) {
        System.out.println("Стъпка " + i);
    }
};

Thread t = new Thread(task);
t.start();

// или направо
new Thread(() ->
    System.out.println("Здравей от нишка"))
    .start();
```

Ламбда изрази: добри практики

- Дръжте ламбдите **кратки** – един-два реда
- Дълга логика → отделен метод + референция към метода
- Предпочитайте стандартните интерфейси пред собствени
- Използвайте описателни имена на параметрите: `student -> student.grade()`
- Не променяйте външно състояние от ламбда – трудно за проследяване
- Внимание с изключенията – стандартните интерфейси не позволяват проверими изключения

05

Анотации и документация

Метаданни в кода и генериране на документация с javadoc

Анотации

- **Метаданни** за кода – не променят пряко логиката
- Записват се с `@`: `@Override`, `@Deprecated`
- Използват се от:
  - **компилатора** – откриване на грешки, потискане на предупреждения
  - **инструменти** – генериране на код и документация
  - **библиотеки по време на изпълнение** – JUnit (`@Test`), Spring (`@Service`)
- Могат да имат параметри: `@SuppressWarnings("unchecked")`

Вградени анотации

Анотация	Предназначение
`@Override`	методът припокрива метод на родител / интерфейс
`@Deprecated`	елементът е остарял и не трябва да се използва
`@SuppressWarnings`	потиска конкретни предупреждения на компилатора
`@FunctionalInterface`	интерфейсът трябва да има точно един абстрактен метод
`@SafeVarargs`	методът с varargs от generic тип е безопасен

Собствени анотации

- Декларират се с `@interface`
- Елементите им изглеждат като методи
- Мета-анотации: `@Retention`, `@Target`
- В практиката по-често **използваме** чужди анотации

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Author {
    String name();
    String date() default "2026";
    int version() default 1;
}

public class Library {
    @Author(name = "Д. Гоцева",
            version = 2)
    public void borrow(String isbn) {
        // ...
    }
}
```

Кога се обработват анотациите?

`@Retention`	Къде съществува	Кой я използва
`SOURCE`	само в изходния код	компилаторът и инструменти: `@Override`, `@SuppressWarnings`
`CLASS`	и в `.class` файла	инструменти за байткод
`RUNTIME`	и по време на изпълнение	библиотеки чрез рефлексия: JUnit, Spring, Jackson

Анотации в популярни библиотеки

```
// JUnit 6 – тестове (лекция 9)
@Test
@DisplayName("Депозит увеличава баланса")
void depositIncreasesBalance() { /* ... */ }

// Jackson – JSON
public record User(@JsonProperty("user_name") String name, int age) { }

// Spring – уеб услуга
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() { return "Здравей!"; }
}

// Jakarta Persistence – таблица в база данни
@Entity
public class Student { @Id Long id; String name; }
```

Документиране с JavaDoc

- Коментари `/** ... */` пред класове, методи и полета
- Тагове: `@param`, `@return`, `@throws`, `@see`, `@since`
- Инструментът `javadoc` генерира HTML документация
- Така е генерирана документацията на JDK

```java
/**
 * Изчислява средноаритметично.
 *
 * @param values оценките, поне една
 * @return средната стойност
 * @throws IllegalArgumentException
 *         ако няма стойности
 * @since 1.2
 */
public static double average(int... values) {
    if (values.length == 0) {
        throw new IllegalArgumentException();
    }
    return Arrays.stream(values)
                 .average().orElseThrow();
}
```

Генериране на документацията

```
$ javadoc -d docs -encoding UTF-8 -charset UTF-8 src/bg/tu/pik3/*.java

Loading source files for package bg.tu.pik3...
Constructing Javadoc information...
Building index for all the packages and classes...
Generating docs/index.html...

B Eclipse:  Project → Generate Javadoc...
```

Добре документираният публичен API е задължителен за библиотеки, които ползват други хора

Обобщение

- Интерфейсът е **договор**; класът може да имплементира много интерфейси
- `default`, `static` и `private` методи в интерфейсите (Java 8–9)
- Абстрактен клас – общо състояние и код; интерфейс – способност
- Вложени класове: статични, вътрешни, локални, анонимни
- **Функционален интерфейс** – един абстрактен метод → ламбда израз
- `java.util.function`: `Function`, `Predicate`, `Consumer`, `Supplier`
- Референции към методи: `String::length`, `System.out::println`
- Анотациите са метаданни; JavaDoc генерира документация

Въпроси за самопроверка

- Каква е разликата между абстрактен клас и интерфейс?
- Защо в Java 8 са добавени default методите?
- Каква е разликата между статичен вложен и вътрешен клас?
- Какво е функционален интерфейс? Дайте три примера от JDK.
- Какво означава „ефективно final“ и защо е важно за ламбда изразите?
- Запишете `s -> s.length()` като референция към метод.
- За какво служи анотацията `@Override`?

Въпроси?

Следваща лекция: Generics и колекции

